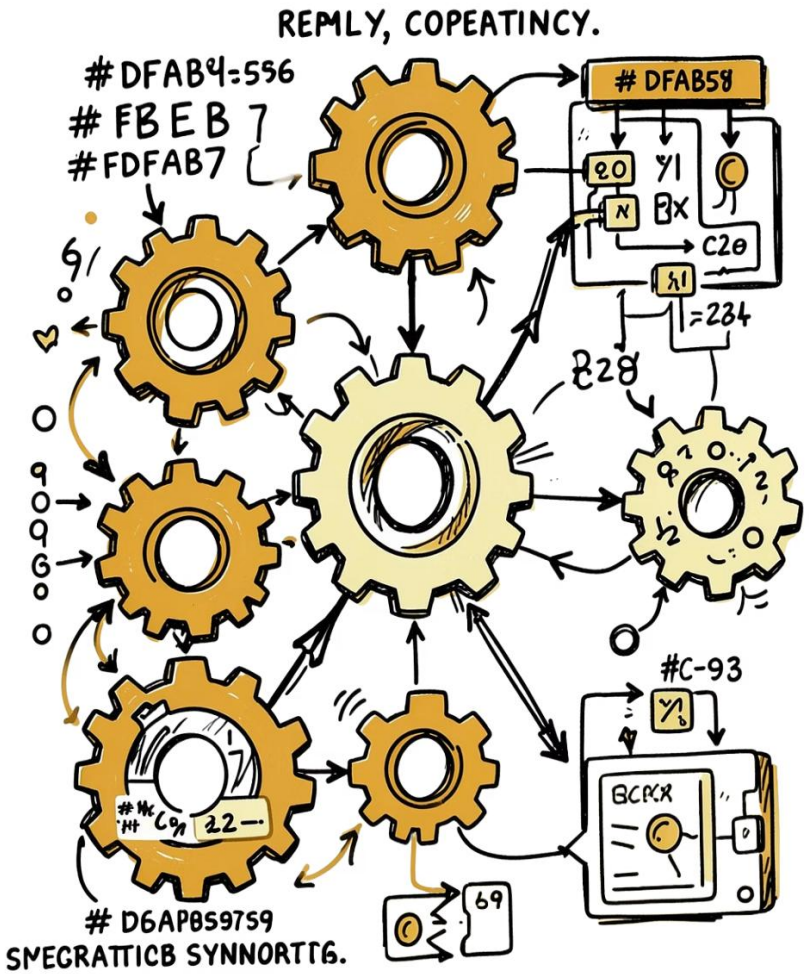
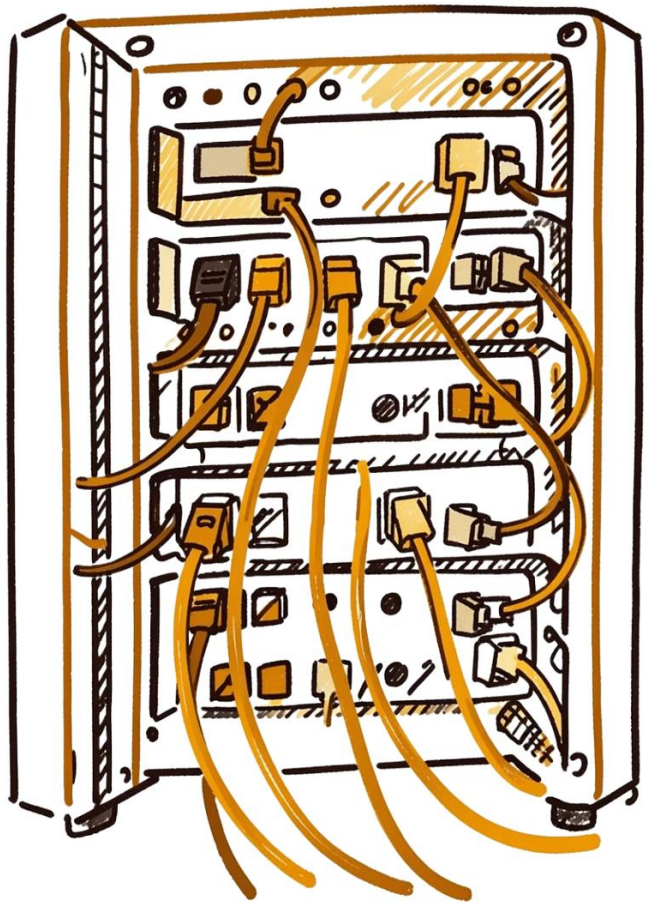


[1]

Priority Ceiling Protocol - Resource Access Protocols

A study of concurrency control mechanisms in real-time systems — from basic semaphores to advanced protocols that bound priority inversion and guarantee schedulability.

PREPARED BY CHIMEZIE DANIEL CHIDI 1230515



[1]

Resources, Critical Sections & Semaphores

Private Resource

Dedicated to a single process.

Shared Resource

Accessible by multiple tasks.

Exclusive Resource

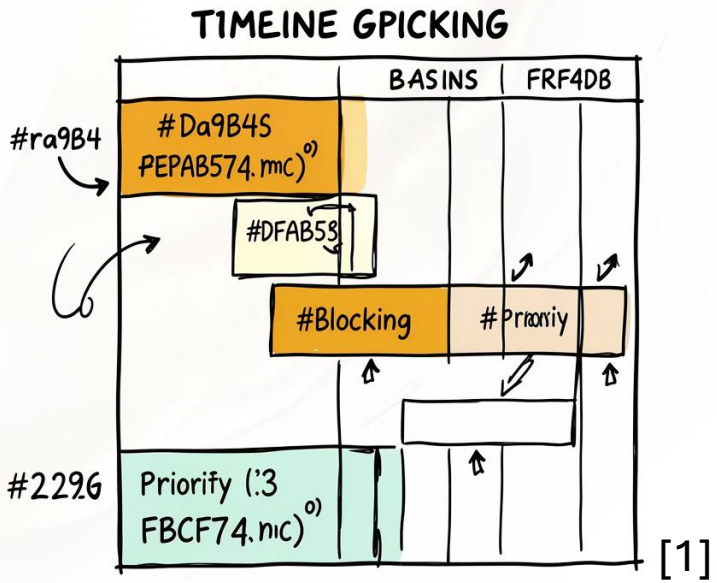
Shared but protected against concurrent access.

A **critical section** is code executed under mutual exclusion. Tasks use **semaphores** via *wait*/*signal* primitives. Blocked tasks enter a **WAIT** state, then return to **READY** (not RUN) when signaled. [2]

Applications

Industry	Example
✈️ Aerospace	Flight Control Computers
🚗 Automotive	ABS, Airbag, Engine Control Units
🩺 Medical	Ventilators, Infusion Pumps, Patient Monitors
🤖 Industrial Automation	Robotic Arms, CNC Machines, Production Lines
🚆 Rail Systems	Train Signaling and Control
🚁 Drones	Flight Stabilization and Navigation
⚡ Power Systems	Substation Protection Relays
🛰️ Space Systems	Satellite Control Software

The Priority Inversion Phenomenon



[1]

What Is It?

A high-priority task is forced to wait for lower-priority tasks holding a shared resource — potentially **unboundedly**.

Classic Scenario

τ_3 holds a resource → τ_1 preempts and blocks → τ_2 (medium priority) preempts τ_3, extending τ_1's wait beyond τ_3's critical section.

⚠ Any intermediate-priority task can prolong blocking, causing critical deadlines to be missed.

Five Resource Access Protocols

01

Non-Preemptive Protocol (NPP)

Disables preemption inside all critical sections.

02

Highest Locker Priority (HLP)

Raises priority to the resource's *ceiling* on entry.

03

Priority Inheritance Protocol (PIP)

Blocking tasks inherit the highest blocked priority.

04

Priority Ceiling Protocol (PCP)

Denies lock if priority ≤ any locked semaphore's ceiling.

05

Stack Resource Policy (SRP)

Blocks at preemption time; supports EDF and stack sharing.

Non-Preemptive & Highest Locker Priority

NPP

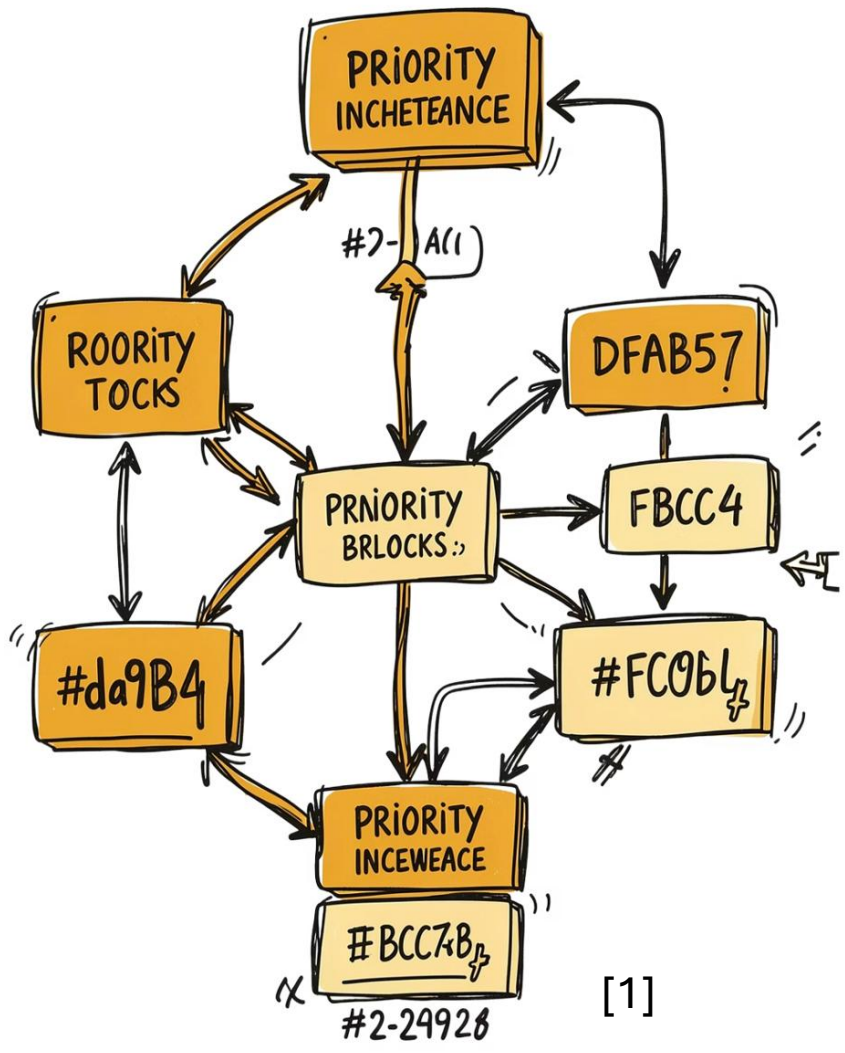
Raises task priority to the system maximum inside any critical section. Simple, but causes **unnecessary blocking** — even high-priority tasks not sharing the resource are blocked.

Max blocking: longest critical section of any lower-priority task.

HLP / IPC

Raises priority to the **resource ceiling** — the highest priority of tasks sharing that resource.

Bounds blocking to **one critical section**. More precise than NPP, but may block before the resource is actually needed.



[1]

Priority Inheritance Protocol (PIP)

Direct Blocking

High-priority task waits for a lower-priority task holding a needed resource.

Push-Through Blocking

Medium-priority task blocked by a low-priority task that inherited a higher priority.

Transitive Inheritance

If τ_3 blocks τ_2 and τ_2 blocks τ_1, then τ_3 inherits τ_1's priority via τ_2.

Under PIP, τ_i can be blocked at most $\alpha_i = \min(l_i, s_i)$ critical sections, where l_i = lower-priority tasks that can block, s_i = distinct blocking semaphores.

PIP: Chained Blocking & Deadlock

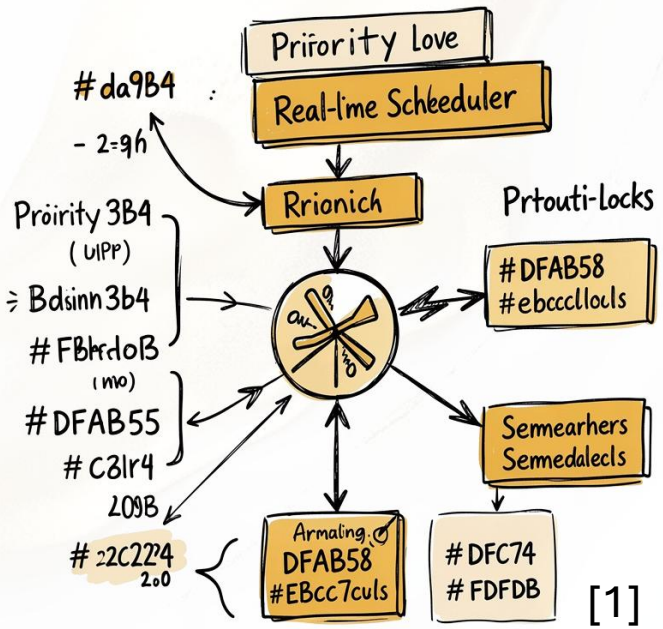
Chained Blocking

If τ_1 accesses *n* semaphores each locked by a different lower-priority task, it can be blocked for **n critical sections** — still bounded, but potentially substantial.

Deadlock

PIP does **not** prevent deadlocks. Two tasks nesting semaphores in reverse order can deadlock. Solution: impose a **total ordering** on semaphore accesses.

Priority Ceiling Protocol (PCP)



[1]

Core Rule

A task may enter a critical section only if its priority is **higher** than the ceiling of all semaphores currently locked by other tasks.

Key Properties

- Blocks at most **one** critical section
- **Prevents deadlocks** by construction
- **Prevents chained blocking**
- Introduces **ceiling blocking** as a third form

Stack Resource Policy (SRP)

Preemption Test

Task τ may run only if $\pi(\tau) > \Pi_s$ (system ceiling). Blocks at **preemption time**, not access time.

Dynamic Priorities

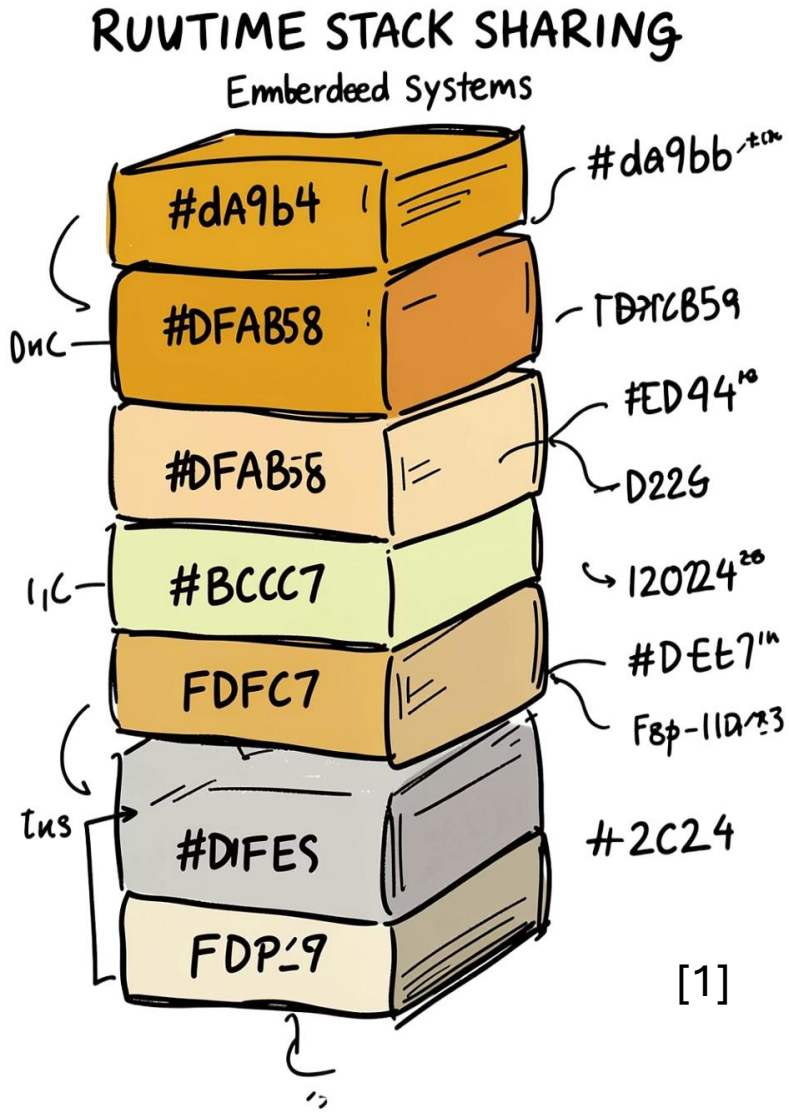
Supports both fixed-priority (RM) and dynamic (EDF) scheduling natively.

Stack Sharing

Tasks share a single stack — saves up to **90%** memory with many tasks across few preemption levels.

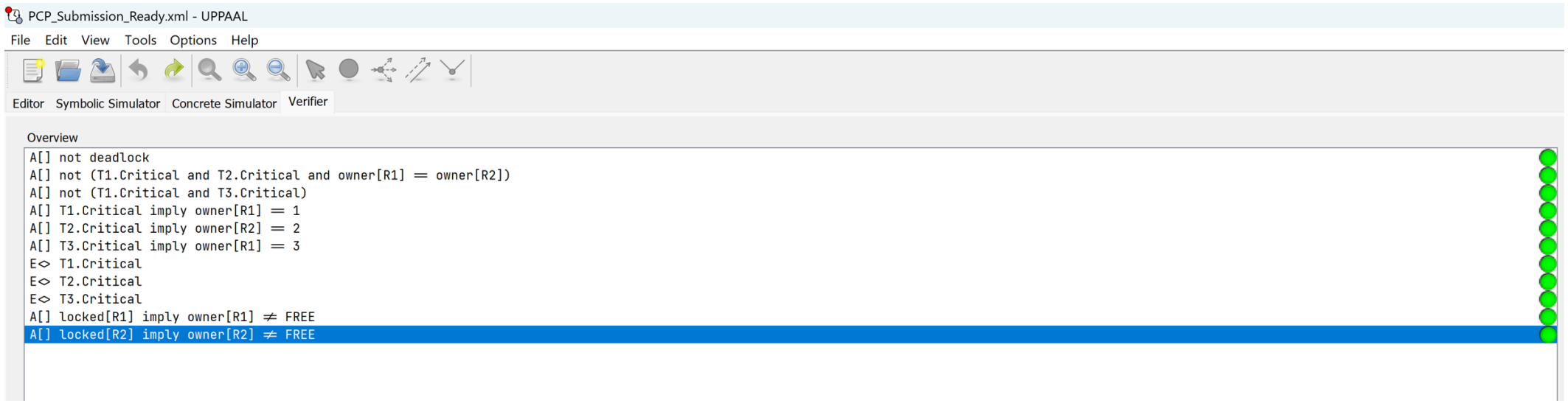
Multi-Unit Resources

Generalizes PCP to resources with multiple concurrent units.



[1]

PCP_UPPAAL Analysis



Protocol	Definition	Blocking	Deadlock Free	Priority Type	Complexity
NPP (Non-Preemptive Protocol)	Disables preemption while inside a critical section.	1 CS	✓	Any	Easy
HLP (Highest Locker Priority)	Task inherits the ceiling priority of the resource it locks.	1 CS	✓	Fixed	Easy
PIP (Priority Inheritance Protocol)	Resource holder inherits the priority of the highest blocked task.	α_i	✗	Fixed	Hard
PCP (Priority Ceiling Protocol)	Task can lock a resource only if its priority exceeds the current system ceiling.	1 CS	✓	Fixed	Medium
SRP (Stack Resource Policy)	Blocks tasks at preemption time using a system ceiling; supports stack sharing.	1 CS	✓	Any (RM/EDF)	Easy

References

[1] Gamma

[2] G. C. Buttazzo, Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications, 3rd ed. New York, NY, USA: Springer, 2011.